



Experiment 7

Student Name: Shivanshu Ranjan

Branch: CSE

Semester: 6th

Subject Name: Full Stack Development – II

UID: 23BCS10193

Section/Group: KRG 3-B

Date of Performance: 16/03/2026

Subject Code: 23CSH-309

1. Aim: To design and implement a **Spring Boot REST API using Layered Architecture**, applying **Inversion of Control (IoC)** and **Dependency Injection (DI)** to build a structured backend for the **HealthHub application**, including CRUD operations, DTO mapping, validation, and global exception handling.

2. Objective:

- Understand the role of **Spring Boot** in backend development
- Configure and run a **basic Spring Boot application**
- Apply **IoC (Inversion of Control)** and **Dependency Injection**
- Use Spring annotations such as **@Autowired**, **@Service**, and **@Repository**
- Implement **Layered Architecture (Controller – Service – Repository)**
- Develop **RESTful APIs using Spring Boot**
- Perform **CRUD operations (Create, Read, Update, Delete)** using HTTP methods
- Implement **DTO mapping** to separate internal data models from API responses
- Apply **validation using Jakarta Validation annotations** like **@NotNull**, **@Email**
- Implement **Global Exception Handling using @ControllerAdvice**
- Build backend services for **HealthHub patient management system**.

3. Implementation / Code:

▪ Tools & Technologies Used:-

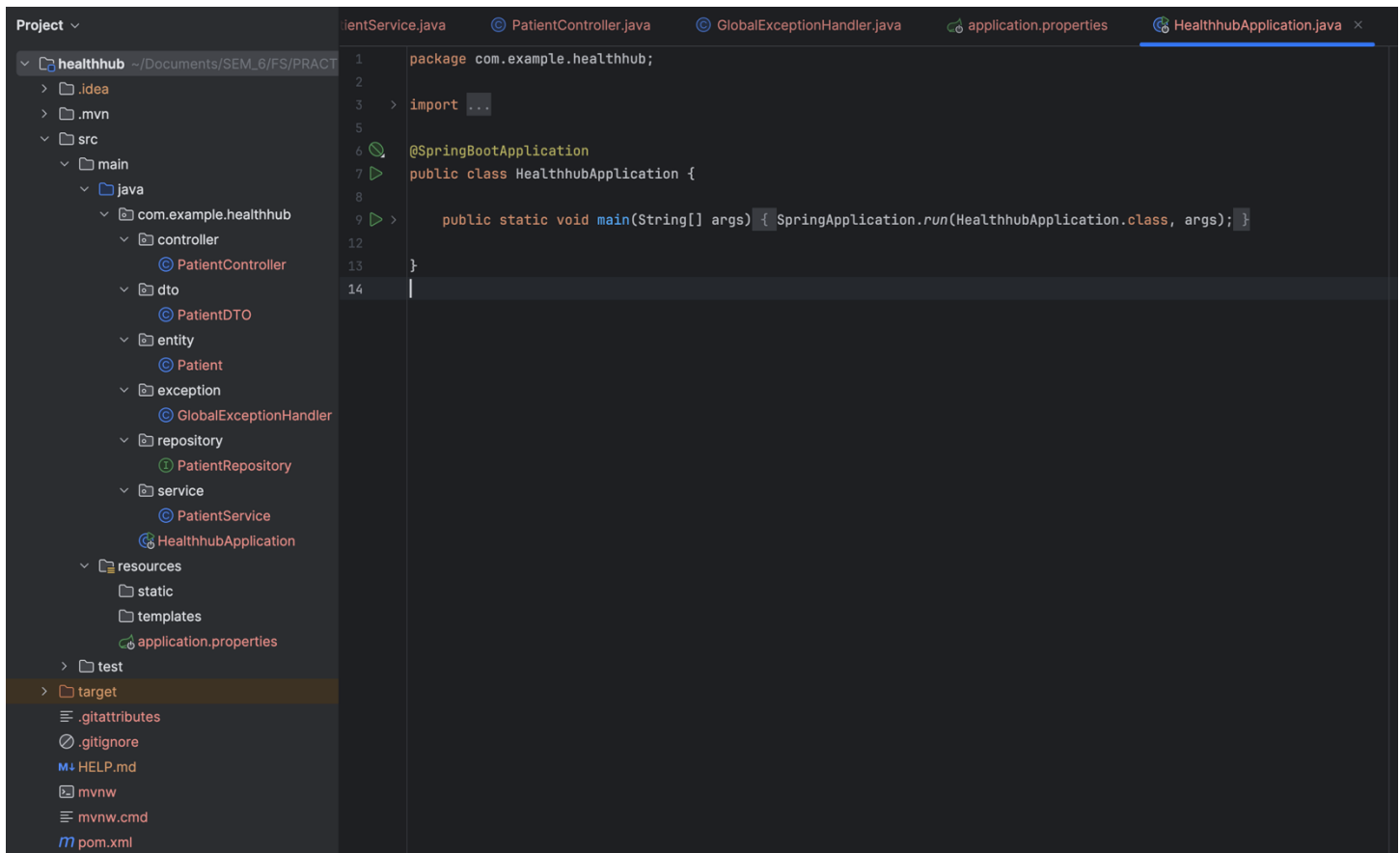
- Java
- Spring Boot
- Spring Web
- Spring Data JPA
- Maven
- IntelliJ IDEA
- Postman (API testing)
- H2 Database

▪ Implementation Description:-

- A **Spring Boot** application named **HealthHub** is created to manage patient data.
- The project follows **Layered Architecture**, separating logic into different layers:

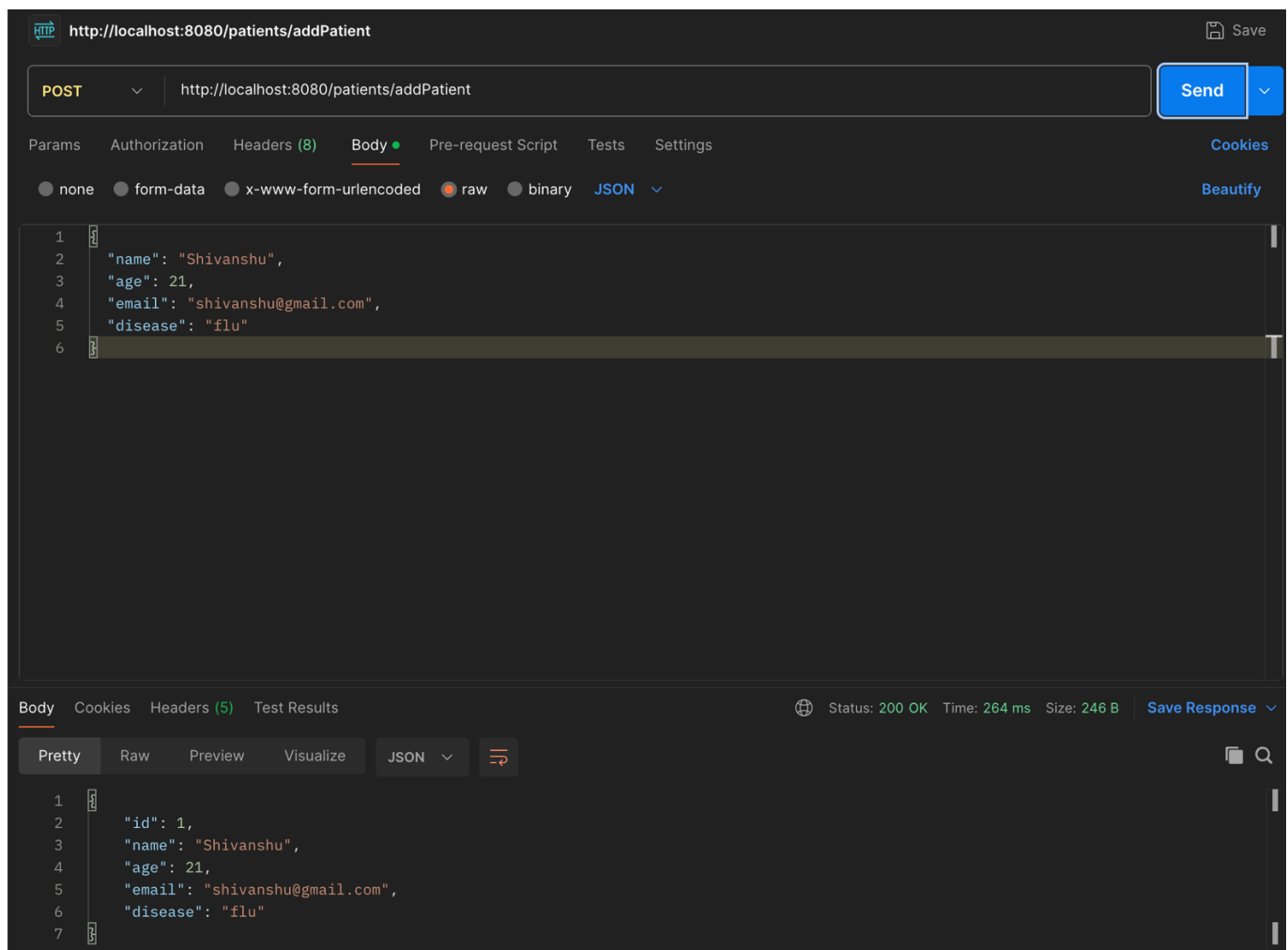
- **Controller Layer**
Handles HTTP requests and responses using REST APIs.
- **Service Layer**
Contains business logic for managing patients.
- **Repository Layer**
Communicates with the database using Spring Data JPA.
- **Entity Layer**
Represents database tables.
- **DTO Layer**
Transfers patient data between API and application.
- **Exception Layer**
Handles errors globally using `@ControllerAdvice`.

▪ Project Structure:-



4. Output:

- Spring Boot application started successfully on **localhost:8080**
- POST API successfully added patient records
- GET API returned the list of patients stored in the database
- GET by ID API returned specific patient details
- DELETE API successfully removed patient data
- Postman confirmed successful API responses with **Status 200 OK**
- Layered architecture ensured separation of concerns and clean backend design





DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

HTTP client interface showing a GET request to `http://localhost:8080/patients/getPatients`. The response is a JSON array of two patient objects, displayed in the Body tab.

Request: `GET http://localhost:8080/patients/getPatients`

Response Body (JSON):

```
1 {
2   {
3     "id": 1,
4     "name": "Shivanshu",
5     "age": 21,
6     "email": "shivanshu@gmail.com",
7     "disease": "flu"
8   },
9   {
10    "id": 2,
11    "name": "Tanya",
12    "age": 21,
13    "email": "tanya@gmail.com",
14    "disease": "flu"
15  }
16 }
```

Status: 200 OK Time: 10 ms Size: 323 B

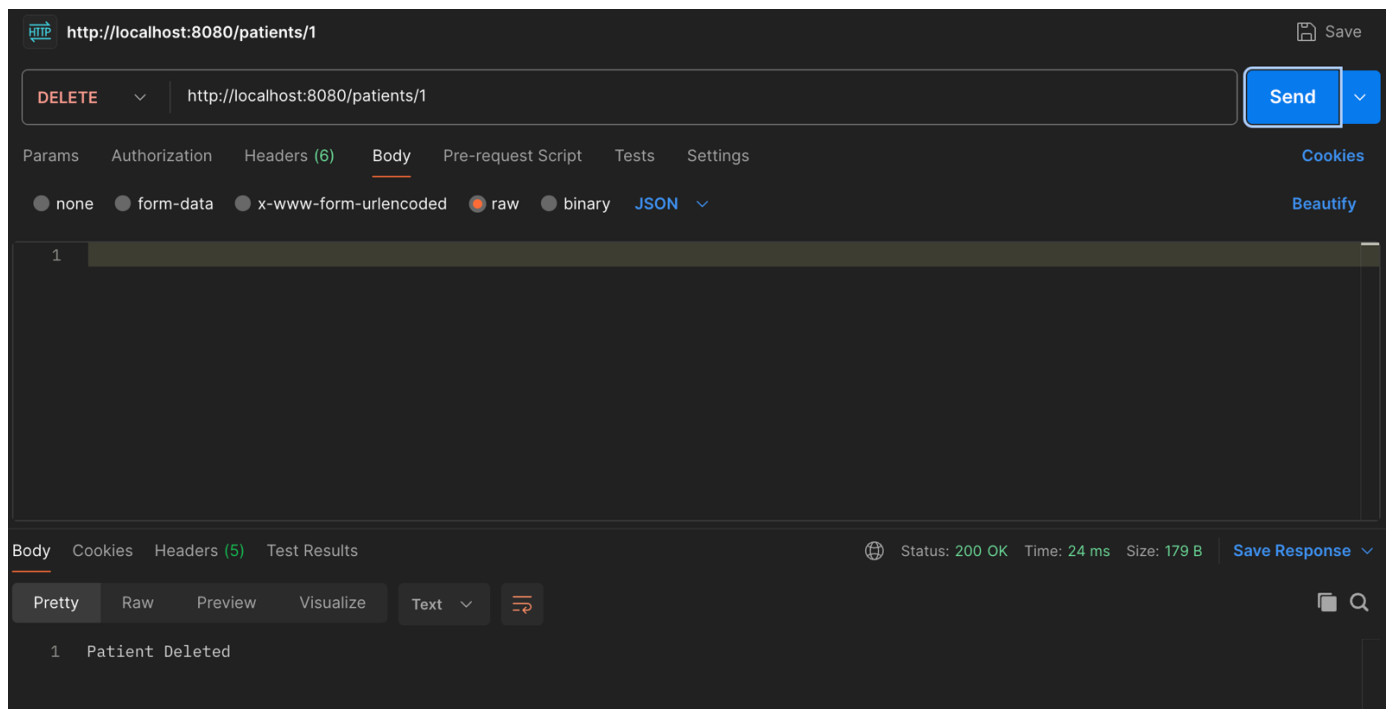
HTTP client interface showing a GET request to `http://localhost:8080/patients/1`. The response is a JSON object representing a single patient, displayed in the Body tab.

Request: `GET http://localhost:8080/patients/1`

Response Body (JSON):

```
1 {
2   "id": 1,
3   "name": "Shivanshu",
4   "age": 21,
5   "email": "shivanshu@gmail.com",
6   "disease": "flu"
7 }
```

Status: 200 OK Time: 8 ms Size: 246 B



5. Learning Outcomes (What I Have Learnt):

- Understanding of **Spring Boot** backend development
- Implementation of **Layered Architecture (Controller, Service, Repository)**
- Usage of **Dependency Injection in Spring Boot**
- Development of **RESTful APIs using HTTP methods**
- Implementation of **CRUD operations in backend applications**
- Understanding of **DTO mapping for API communication**
- Applying **validation annotations for data integrity**
- Handling errors globally using **Spring Exception Handling**
- Testing APIs using **Postman**